RabbitMQ : envoyer, traiter, router

TP étudiant · IWA / AWI · En binôme · 1 heure

Votre mission. Une boutique doit envoyer des e-mails et informer plusieurs services lorsqu'une commande change. Vous allez utiliser RabbitMQ pour faire communiquer ces programmes sans les faire travailler tous au même moment.

À la fin de la séance, vous devrez pouvoir : envoyer et recevoir un message ; expliquer quand il est retiré de la queue ; répartir des tâches entre deux workers ; observer une reprise après panne ; choisir les destinataires d'un événement.

Cinq mots pour démarrer

Terme	À quoi sert-il ?
Producer	Programme qui publie un message.
Exchange	Reçoit les publications et les dirige vers des queues.
Queue	Conserve des messages en attente de traitement.
Consumer / worker	Programme qui reçoit et traite des messages.
Ack	Confirmation envoyée à RabbitMQ après le traitement.

```
Producer  -->  Exchange  -->  Queue  -->  Consumer
                  |
           règle de routage (binding)
```

Une **routing key** accompagne la publication. Une **binding key** décrit une règle de liaison entre un exchange et une queue. Une connexion relie votre programme au broker ; un channel permet d'y effectuer les opérations de messagerie.

Votre parcours

Étape	Durée
Comprendre le scénario et se répartir les rôles	5 min
1. Manipuler un message dans l'interface	8 min
2. Relier un producer et un consumer	10 min
3. Partager des tâches et observer une panne	15 min
4. Router des événements	17 min
5. Expliquer ce que vous avez observé	5 min

Organisation. Choisissez un seul langage. Une personne manipule, l'autre prédit et note ; inversez les rôles après la partie 2. Les installations se font avant la séance. Les bonus sont hors des 60 minutes.

Binôme : ..... Langage : .....

Avant la séance : préparer le poste

Cette préparation ne compte pas dans l'heure de TP

1. Récupérer et installer le kit

Décompressez **TP-RabbitMQ-kit-etudiant.zip**. Gardez le PDF ouvert et choisissez uniquement le dossier `python`, `node` ou `java`. Les fiches des pages 9 à 11 donnent les commandes de votre langage. Sous Windows, utilisez PowerShell pour les commandes de ce support.

Parcours	Prérequis	Fiche
Python	Python 3.10+ et pip	Page 9
Node.js	Node.js 22+ et npm ; JavaScript pour éviter la configuration TypeScript	Page 10
Java	JDK 17+ et Maven ; Spring AMQP déjà configuré	Page 11

Installez les dépendances et faites la vérification indiquée dans votre fiche. Utilisez une seule machine et un seul broker par binôme. En cas de blocage d'installation, faites-le résoudre avant la séance par l'enseignant.

2. Démarrer Docker, puis RabbitMQ

Vérifiez que Docker Desktop ou OrbStack fonctionne. Depuis le dossier racine du kit, où se trouve `compose.yaml`, créez et démarrez le conteneur local :

```
docker compose up -d
```

Si ce conteneur existe déjà, utilisez seulement :

```
docker start tp-rabbitmq
```

Le premier téléchargement peut prendre plusieurs minutes. Ouvrez ensuite **http://localhost:15672** ; identifiant `guest`, mot de passe `guest`. Le port **15672** sert à l'interface ; **5672** sert à vos programmes.

Point de contrôle. L'onglet Overview s'affiche. Si la connexion échoue, attendez un peu et consultez `docker logs tp-rabbitmq`. Le temps d'attente dépend de la machine : 20 secondes ne sont pas une garantie.

3. Repérer le travail à faire

Les fichiers de connexion et les workers sont fournis. Complétez seulement les zones **TODO H**, **TODO T1** et **TODO T2**. Remplacez l'instruction qui lève l'erreur TODO par votre code. Ces erreurs sont volontaires : elles ne signalent pas une installation cassée.

Les appels API utiles figurent dans la fiche de votre langage. Ne modifiez pas les noms de queues ou les réglages fournis pendant les premières expériences.

1. Manipuler un message

8 minutes · Sans code

But. Comprendre ce que signifie « lire » puis « acquitter » un message.

Prédire, puis manipuler

1. Dans **Queues and Streams**, ouvrez **Add a new queue**. Créez `demo.test`, type **Classic**, durabilité **Durable**. Ouvrez sa fiche.

2. Dans **Publish message**, saisissez `Salut` dans **Payload**, puis publiez. Attendez que le compteur soit actualisé.

3. Avant de lire, notez votre prédiction : si on remet le message dans la queue après lecture, pourra-t-on le relire ?

4. Dans **Get messages**, choisissez **Ack Mode = Nack message requeue true**, **Messages = 1**, puis **Get Message(s)**. Lisez le contenu. Renouvelez cette lecture une fois.

5. Passez à **Automatic ack**, puis cliquez sur **Get Message(s)**. Cliquez encore une fois : que répond l'interface ?

Attention à l'affichage. Les métriques sont calculées et actualisées avec un délai. Après une action, patientez quelques secondes, puis vérifiez de nouveau. Un compteur momentanément ancien ne prouve pas que l'opération a échoué. Le texte de la réponse à une lecture est également une observation utile.

Garder une trace

Action	Ma prédiction	Mon observation après actualisation
Lecture avec remise en queue	.....	.....
Deuxième lecture avec remise	.....	.....
Lecture avec Automatic ack	.....	.....
Lecture suivante	.....	.....

Repères. `Ready` compte les messages disponibles pour livraison. `Unacked` compte ceux déjà livrés dont RabbitMQ attend encore l'acquittement. Une lecture depuis l'interface ne permet pas forcément de voir un état Unacked durable : vous l'observerez pendant un traitement lent en partie 3.

Q1. Quelle différence faites-vous entre lire un message et confirmer son traitement ? Appuyez-vous sur vos observations.

Point de contrôle. Vous avez relu le même contenu avec remise en queue, puis testé une lecture après acquittement. Conservez votre conclusion pour le bilan.

2. Envoyer et recevoir depuis du code

10 minutes · TODO H

But. Faire communiquer deux programmes : un producer envoie un message dans `hello`, un consumer l'affiche.

```
hello-send  -->  exchange par défaut  -->  hello  -->  hello-consumer
```

L'exchange par défaut porte le nom vide `''`. Sa routing key désigne le nom de la queue cible. La queue est déjà déclarée dans les squelettes : votre travail porte sur la publication.

Compléter et essayer

1. Ouvrez `hello-send.py`, `hello-send.js` ou la méthode `publierHello()` de `TpApplication.java` selon votre langage.

2. Remplacez **TODO H** pour publier `Bonjour RabbitMQ !` vers `hello`. Utilisez l'aide API de votre fiche.

3. **Terminal P : producer.** Lancez l'envoi avec la commande de votre fiche. Aucun consumer ne doit encore tourner. Dans l'interface, ouvrez `hello` et relevez `Ready` après actualisation.

4. **Terminal C : consumer.** Lancez le consumer fourni. Vérifiez le texte reçu dans son terminal, puis les compteurs de `hello`.

5. Gardez C ouvert. Depuis P, relancez deux fois le producer. Notez combien de nouveaux affichages apparaissent.

6. Arrêtez le consumer avec **Ctrl+C** avant de passer à la partie 3.

Observer

Moment	Ready après actualisation	Nombre de nouveaux affichages
Après le premier envoi, consumer arrêté	.....	.....
Après le démarrage du consumer	.....	.....
Après deux nouveaux envois	.....	.....

Q2. Le producer a-t-il eu besoin d'attendre le lancement du consumer pour déposer son premier message ? Quelle partie du système l'a conservé entre-temps ?

Q3. Repérez la déclaration de `hello` dans les deux programmes. Pourquoi peut-on vouloir déclarer la même queue des deux côtés ? Les paramètres doivent être identiques.

Point de contrôle. Chaque publication testée correspond à un affichage dans le consumer et vous savez montrer la queue dans l'interface.

3. Partager des tâches et reprendre après une panne

15 minutes · Worker fourni · Prefetch fixé à 1

But. Observer deux workers qui se partagent une queue, puis le devenir d'une tâche interrompue. Une tâche représente un e-mail ; chaque point . ajoute une seconde de traitement.

3.1. Deux workers, une queue

1. Lisez le worker fourni. Repérez : la queue `tasks.email`, le réglage **prefetch=1**, l'attente simulant le travail et le moment de l'ack. En Java, lisez le commentaire sur l'acquittement effectué par Spring.

2. **Terminaux A et B :** démarrez un worker dans chacun avec le prefetch à 1. Attendez que les deux soient prêts ; vérifiez dans la fiche de `tasks.email` que deux consumers sont connectés.

3. **Terminal P :** envoyez les six tâches avec la commande de votre fiche. Regardez `Ready` et `Unacked` pendant leur traitement.

4. Notez les messages traités par A et B. Attendez la fin : `Ready=0` et `Unacked=0` après actualisation.

Worker	Numéros des tâches traitées	Ce que j'ai observé
A	.....	.....
B	.....	.....

Le prefetch limite ici le nombre de messages livrés à chaque worker et encore non acquittés. Les deux workers exécutent chacun un seul traitement à la fois. La répartition exacte et la durée totale peuvent varier : vous n'avez pas à reproduire une liste de numéros imposée.

3.2. Interrompre un traitement

1. Arrêtez B. Vérifiez qu'il ne reste qu'un consumer connecté.

2. Depuis P, envoyez la tâche **panne** de votre fiche : elle dure 30 secondes pour vous laisser le temps de manipuler.

3. Dès que A affiche **Début**, observez la fiche de la queue. Interrompez A avant **Terminé**. Python et Node : Ctrl+C. Java : utilisez l'arrêt brutal détaillé dans votre fiche ; Ctrl+C peut laisser finir un traitement.

4. Attendez que RabbitMQ détecte la fermeture, puis que les compteurs s'actualisent. Notez `Ready` et `Unacked` sans démarrer d'autre worker.

5. Redémarrez B. Attendez la fin du traitement et relevez l'indication **redélivré** dans le terminal.

Q4. Les deux workers reçoivent-ils chacun une copie de chaque tâche, ou se les partagent-ils ? Justifiez avec votre tableau.

Q5. À quoi sert le prefetch de 1 ? Que risquerait-on en acquittant avant la fin du traitement ?

Point de contrôle. Vous avez observé une tâche interrompue, sa remise à disposition, puis son traitement par un nouveau worker. Arrêtez les workers une fois l'expérience terminée.

4. Router des événements vers plusieurs services

17 minutes · TODO T1 et TODO T2

But. Faire parvenir les événements utiles aux services Stock, Notifications et Audit. Ils utilisent chacun leur queue, reliée à l'exchange topic `events`.

Service	Queue	Binding key à utiliser
Stock	stock.queue	commande.*
Notifications	notifs.queue	commande.creee
Audit	audit.queue	#

Dans un motif topic, les segments sont séparés par des points : `*` remplace exactement un segment ; `#` remplace zéro, un ou plusieurs segments. Une binding key sans joker demande une correspondance exacte.

4.1. Compléter et prédire

1. Dans le consumer topic, complétez **TODO T1** : créer le binding pour chaque clé fournie en argument. La déclaration de l'exchange et de la queue est déjà donnée.

2. Dans le publisher topic, complétez **TODO T2** : publier le contenu avec la routing key courante. Remplissez d'abord les colonnes de prédiction ci-dessous, sans lancer le publisher.

3. Lancez les trois consumers avec votre fiche : un terminal Stock, un Notifications et un Audit. Attendez leurs messages d'attente et vérifiez les bindings de `events` dans l'interface.

4. Depuis un quatrième terminal, publiez le lot par défaut. Comparez chaque prédiction aux affichages. Comptez les réceptions, pas seulement les lignes de démarrage.

Événement publié	Destinataires prédits	Destinataires observés
commande.creee	.....	.....
commande.annulee	.....	.....
paiement.refuse	.....	.....

4.2. Étendre, puis arrêter

5. Ajoutez un consumer **livraison**, binding key `commande.#`. Prédisez les destinataires de `commande.creee.urgent`, puis publiez cet événement avec la commande de votre fiche.

6. Arrêtez tous les consumers. Les queues doivent rester présentes. Publiez de nouveau `commande.creee` ; relevez les messages en attente. Redémarrez ensuite les consumers pour observer leur réception.

Q6. Pourquoi avoir vérifié l'existence des queues et des bindings avant la première publication ? Que faudrait-il tester pour savoir ce qui arrive à une publication sans queue correspondante ?

Q7. Vos prédictions étaient-elles correctes ? Expliquez une différence entre `commande.*` et `commande.#` avec l'événement de l'étape 5.

Point de contrôle. Vous avez un tableau complété, un essai avec Livraison et une observation de publication alors que les consumers sont arrêtés.

5. Expliquer ce que vous avez observé

5 minutes · Mini rendu du binôme

Produisez une courte note, une page maximum. Vous pouvez répondre directement sous ces questions ou dans un fichier texte. Il n'est pas demandé de recopier le code fourni.

1. Deux architectures, deux usages. Avec un exemple du TP, expliquez la différence entre deux workers sur une même queue et deux services utilisant chacun leur queue liée au même exchange.

.....
.....

2. Une panne pendant le travail. Décrivez les étapes observées avant l'interruption, après la fermeture de la connexion et après le redémarrage d'un worker. Pourquoi faut-il prévoir qu'un traitement puisse être exécuté de nouveau ?

.....
.....

3. Une règle de routage. Indiquez les destinataires que vous avez observés pour `commande.creee.urgent` et justifiez-les avec les binding keys.

.....
.....

4. Une difficulté rencontrée. Décrivez un blocage ou une prédiction fausse, puis la vérification qui vous a permis d'avancer.

.....

Avant de rendre

- Vos tableaux distinguent les prédictions et les observations.
- Vous avez identifié le moment de l'acquittement dans votre parcours.
- Vous avez observé une redélivrance après interruption.
- Vos conclusions s'appuient sur le terminal et sur l'interface, en tenant compte du délai des métriques.

En cas de blocage pendant la séance

Connexion refusée : vérifiez `docker ps`, puis `docker logs tp-rabbitmq`. Vérifiez le port 5672 et que vous êtes dans le bon dossier.

Erreur TODO : complétez la zone indiquée et remplacez l'instruction qui lève l'erreur. **Aucun affichage :** vérifiez la queue, les bindings et le nombre de consumers ; un message peut être reçu par un ancien programme encore ouvert.

PRECONDITION_FAILED : une queue ou un exchange existe avec d'autres paramètres. Vérifiez son nom et sa déclaration avec l'enseignant avant de supprimer quoi que ce soit. **Jokers :** gardez `*` et `#` entre guillemets dans les commandes.

Bonus : aller plus loin

Facultatif · Hors des 60 minutes · Choisissez une expérience

A. Comparer deux réglages de prefetch · 10 à 15 min

Arrêtez les workers et assurez-vous que `tasks.email` est vide, sans message non acquitté. Relancez A et B avec **prefetch=6** ; chaque worker peut alors recevoir jusqu'à six messages non acquittés. Attendez deux consumers, puis envoyez le lot des six tâches. Relevez la répartition et le temps entre le premier début et la dernière fin pour chaque worker.

Attendez la fin complète. Arrêtez-les, relancez-les avec **prefetch=1** et renvoyez le même lot. Comparez. Le worker Node fourni impose un traitement successif : ne remplacez pas sa chaîne de promesses par plusieurs temporisateurs indépendants.

À expliquer : quel réglage permet aux workers de réserver beaucoup de tâches à l'avance ? Vos observations dépendent-elles seulement du nombre de messages ou aussi de leur durée ? Ne supposez pas une répartition identique à celle du binôme voisin.

B. Observer la durabilité · 8 à 10 min

Arrêtez tous les workers. Envoyez le lot **durable** de trois messages, puis vérifiez qu'ils sont dans `tasks.email`. Exécutez :

```
docker restart tp-rabbitmq
```

Attendez que le broker et l'interface répondent. Comparez les compteurs avant et après, puis redémarrez un worker. Repérez dans le code la durabilité de la queue et la persistance des messages.

À expliquer : que montre cet essai de redémarrage ? Quelles limites aurait une conclusion disant que les messages ne peuvent jamais être perdus ? Les confirmations de publication, déjà utilisées par le sender de tâches fourni, seront approfondies dans une autre séance. Un redémarrage du conteneur n'est pas une suppression de ses données.

C. Isoler un message rejeté · 10 à 15 min · Interface

1. Créez un exchange **direct**, durable, `demo.dlx`, puis une queue Classic durable `demo.dlq`. Dans l'exchange, liez cette queue avec la clé `dead`.

2. Créez une nouvelle queue Classic durable `demo.work`. Dès sa création, renseignez les arguments String `x-dead-letter-exchange = demo.dlx` et `x-dead-letter-routing-key = dead`.

3. Publiez un message dans `demo.work`. Dans Get messages, choisissez **Reject requeue false**. Cherchez ensuite ce message dans `demo.dlq` ; observez ses en-têtes, notamment `x-death`.

4. Variante : créez `demo.work.ttl` avec les deux mêmes arguments et `x-message-ttl = 10000`, de type **Number**. Publiez sans consumer, attendez plus de 10 secondes, puis observez la DLQ.

À expliquer : quelle différence entre rejeter un message et le remettre dans sa queue ? Une panne suffit-elle à l'envoyer automatiquement en DLQ ? Quelles règles faudrait-il prévoir pour limiter des tentatives répétées ?

Fiche Python

À utiliser seulement si votre binôme a choisi Python

Avant la séance. Ouvrez un terminal dans le dossier `python` du kit.

```
python3 -m venv .venv
# macOS / Linux :
source .venv/bin/activate
# Windows PowerShell : à la place de la ligne précédente
.venv\Scripts\Activate.ps1
python -m pip install -r requirements.txt
python -c "import pika; print(pika.__version__)"
```

Sous Windows, utilisez `py -3` à la place de `python3` si nécessaire. Si PowerShell bloque l'activation, utilisez directement `.venv\Scripts\python.exe` à la place de `python`, sans modifier la politique de sécurité du poste. Dans chaque nouveau terminal, ouvrez le même dossier et activez l'environnement, ou utilisez le chemin direct de son Python.

Commandes pendant le TP

Action	Commande
Envoyer Hello World	python hello-send.py
Écouter Hello World	python hello-consumer.py
Worker A ou B (partie 3)	python task-worker.py 1
Bonus : worker prefetch à 6	python task-worker.py 6
Consumer Stock	python topic-consumer.py stock 'commande.*'
Consumer Notifications	python topic-consumer.py notifs 'commande.creee'
Consumer Audit	python topic-consumer.py audit '#'
Consumer Livraison	python topic-consumer.py livraison 'commande.#'
Publier les trois événements par défaut	python topic-publish.py
Publier l'événement urgent	python topic-publish.py 'commande.creee.urgent'
Republier après arrêt des consumers	python topic-publish.py 'commande.creee'

Lot de six tâches, à saisir sur une seule ligne :

```
python task-send.py 'Email 1.....' 'Email 2.' 'Email 3.....' 'Email 4.' 'Email 5.....' 'Email 6.'
```

Tâche de panne (30 secondes), puis lot durable pour le bonus :

```
python task-send.py 'Email 7.............................'
python task-send.py 'Email A' 'Email B' 'Email C'
```

Aide API pour les TODO

• **H et T2 :** `ch.basic_publish(exchange=..., routing_key=..., body=...)`. Le contenu peut être encodé en bytes avec `.encode()`.

• **T1 :** `ch.queue_bind(queue=..., exchange=..., routing_key=...)`. Pour un binding, le paramètre `routing_key` contient votre binding key.

• **Lecture du worker :** `ch.basic_ack(delivery_tag=...)` acquitte la livraison après le traitement. `auto_ack=False` conserve cet acquittement explicite.

Fiche Node.js

À utiliser seulement si votre binôme a choisi Node.js

Avant la séance. Ouvrez un terminal dans le dossier `node` du kit. Le kit utilise JavaScript et les modules ES ; il ne nécessite pas de configuration TypeScript.

```
node --version
npm ci
node -e "import('amqplib').then(() => console.log('amqplib prête'))"
```

Un fichier de verrouillage des dépendances est fourni. Dans chaque nouveau terminal, ouvrez le même dossier `node`. Les squelettes peuvent être lus dans un projet TypeScript, mais les commandes ci-dessous visent les fichiers JavaScript fournis.

Commandes pendant le TP

Action	Commande
Envoyer Hello World	node hello-send.js
Écouter Hello World	node hello-consumer.js
Worker A ou B (partie 3)	node task-worker.js 1
Bonus : worker prefetch à 6	node task-worker.js 6
Consumer Stock	node topic-consumer.js stock 'commande.*'
Consumer Notifications	node topic-consumer.js notifs 'commande.creee'
Consumer Audit	node topic-consumer.js audit '#'
Consumer Livraison	node topic-consumer.js livraison 'commande.#'
Publier les trois événements par défaut	node topic-publish.js
Publier l'événement urgent	node topic-publish.js 'commande.creee.urgent'
Republier après arrêt des consumers	node topic-publish.js 'commande.creee'

Lot de six tâches, à saisir sur une seule ligne :

```
node task-send.js 'Email 1.....' 'Email 2.' 'Email 3.....' 'Email 4.' 'Email 5.....' 'Email 6.'
```

Tâche de panne (30 secondes), puis lot durable pour le bonus :

```
node task-send.js 'Email 7.............................'
node task-send.js 'Email A' 'Email B' 'Email C'
```

Aide API pour les TODO

• **H :** `ch.sendToQueue(nom, contenu)` publie via l'exchange par défaut.

• **T1 :** `await ch.bindQueue(queue, exchange, cle)` crée une liaison.

• **T2 :** `ch.publish(exchange, cle, contenu)` publie un événement. Le contenu est un `Buffer`.

• **Lecture du worker :** `await traiter(msg)` finit avant `ch.ack(msg)`. La chaîne `suite` maintient un traitement à la fois. Un callback simplement déclaré `async` ne suffit pas à imposer cette limite.

Conservez les appels `waitForConfirms()` fournis avant la fermeture des publishers : le booléen renvoyé par une publication n'est pas une confirmation du broker.

Fiche Java / Spring AMQP

À utiliser seulement si votre binôme a choisi Java

Avant la séance. Dans le dossier `java` du kit :

```
java -version
mvn -q -DskipTests package
java -jar target/tp-rabbitmq-1.0.0.jar
```

La dernière commande affiche les rôles disponibles et se termine. Le projet n'embarque pas de serveur web : les workers n'utilisent pas de port 8080. Après une modification du code, relancez la compilation ci-dessus, puis redémarrez les programmes concernés.

Commandes pendant le TP

Chaque commande ci-dessous s'exécute dans le dossier `java` et commence par `java -jar target/tp-rabbitmq-1.0.0.jar`, suivi des arguments indiqués. Un terminal distinct par consumer ou worker.

Action	Arguments à ajouter à la commande java -jar
Envoyer Hello World	--tp.role=hello-send
Écouter Hello World	--tp.role=hello-consumer
Worker A ou B	--tp.role=task-worker --tp.prefetch=1
Bonus : worker prefetch à 6	--tp.role=task-worker --tp.prefetch=6
Envoyer les six tâches	--tp.role=task-send --tp.lot=six
Tâche panne de 30 secondes	--tp.role=task-send --tp.lot=panne
Lot durable de trois messages	--tp.role=task-send --tp.lot=durable
Consumer Stock	--tp.role=topic-consumer --tp.name=stock '--tp.keys=commande.*'
Consumer Notifications	--tp.role=topic-consumer --tp.name=notifs --tp.keys=commande.creee
Consumer Audit	--tp.role=topic-consumer --tp.name=audit '--tp.keys=#'
Consumer Livraison	--tp.role=topic-consumer --tp.name=livraison '--tp.keys=commande.#'
Publier le lot de trois événements	--tp.role=topic-publish
Publier l'événement urgent	--tp.role=topic-publish --tp.events=commande.creee.urgent
Republier après arrêt des consumers	--tp.role=topic-publish --tp.events=commande.creee

Exemple complet pour le worker :

```
java -jar target/tp-rabbitmq-1.0.0.jar --tp.role=task-worker --tp.prefetch=1
```

Pour plusieurs binding keys sur un consumer, séparez-les par des virgules dans `--tp.keys`. Le prefetch est défini par le code fourni : il ne dépend pas de la valeur par défaut de Spring.

Simuler une panne. Le programme affiche son PID. Arrêtez uniquement ce worker : macOS/Linux, `kill -9 PID` ; PowerShell, `Stop-Process -Id PID -Force`. Remplacez PID par le nombre affiché. Pour les arrêts ordinaires, utilisez Ctrl+C.

Aide API. H/T2 : `rt.convertAndSend(exchange, cle, texte)`. T1 : construire un `Binding` avec `BindingBuilder`, puis le déclarer avec `admin.declareBinding(...)`. Vous pouvez aussi utiliser le constructeur `Binding(destination, DestinationType.QUEUE, exchange, cle, null)`.

Spring est configuré en **AcknowledgeMode.AUTO** : il acquitte après le retour normal du traitement. Ce mode ne correspond pas à un acquittement immédiat du protocole AMQP.

Repères pratiques et ressources

À garder pour refaire les expériences

Recommencer sans mélanger les essais

Avant de relancer un lot de tâches, attendez la fin des traitements et vérifiez `Ready=0`, `Unacked=0`. Arrêtez les anciens workers avant d'en relancer. Un message déjà en attente peut arriver dès le démarrage d'un consumer.

Les queues et les bindings du parcours topic restent présents après l'arrêt des programmes. Relancer un consumer avec une nouvelle binding key ne supprime pas ses anciennes liaisons. Pour tester une autre règle, utilisez un nouveau nom de service ou demandez à l'enseignant comment retirer le binding concerné.

Si un premier essai a laissé des messages en attente, terminez leur lecture avant de mesurer un nouveau lot. Ne purgez une queue qu'après avoir vérifié qu'elle ne contient que vos données de TP et que leur perte n'affecte aucune expérience en cours.

Un problème d'environnement ?

Symptôme	Vérification à faire
Le nom tp-rabbitmq existe déjà	Utiliser docker start tp-rabbitmq ; ne pas recréer le conteneur.
Port 5672 ou 15672 occupé	Vérifier avec l'enseignant quel broker fonctionne déjà. Si un mapping change, adapter aussi les programmes ou l'adresse web.
Import pika ou amqplib introuvable	Vérifier le dossier courant, l'environnement Python ou npm ci.
Le programme Java affiche encore un TODO supprimé	Recompiler le jar et relancer la nouvelle version.
Les résultats diffèrent du premier essai	Chercher les anciens consumers, messages et bindings avant de modifier le code.

Pour la prochaine séance

Un acquittement concerne une livraison, pas l'unicité de l'action métier. Un traitement peut être exécuté de nouveau : on cherchera alors à le rendre **idempotent**, c'est-à-dire à éviter qu'une répétition produise un effet métier supplémentaire non souhaité.

La durabilité, la persistance et les confirmations de publication répondent à des problèmes différents. Le bonus de redémarrage illustre une situation précise ; les garanties de production demandent d'autres choix et vérifications.

Ressources officielles

Ces ressources documentent les API et les mécanismes ; vos prédictions et vos observations restent à réaliser pendant le TP.

• Tutoriels RabbitMQ : https://www.rabbitmq.com/tutorials

• Acquittements et confirmations : https://www.rabbitmq.com/docs/confirms

• Routage topic : https://www.rabbitmq.com/tutorials/tutorial-five-python

• Dead letter exchanges : https://www.rabbitmq.com/docs/dlx

• API amqplib : https://amqp-node.github.io/amqplib/channel_api.html

• API Pika : https://pika.readthedocs.io/en/stable/modules/channel.html

• Spring AMQP : https://docs.spring.io/spring-amqp/reference/

Fin de séance. Arrêtez vos consumers, puis le broker avec `docker stop tp-rabbitmq`. Le conteneur est conservé ; vous pourrez le relancer avec `docker start tp-rabbitmq`.